Lab 1: Memory Allocator 实验报告

潘孝圆　24300240128　2026 年 10 月 30 日

本次修改 `src/kernel/mem.c`，实现整页分配、释放，以及按 8 字节分档的小块分配器。

任务 1：接入初始化流程

沿用课程 Lab1 的 `main.c`：CPU 0 清零 BSS、初始化串口与打印服务，调用 `kinit()` 后才放行其他核心；各核心随后进入 `idle_entry()` 执行 `kalloc_test()`。保留原启动流程和测试代码。

任务 2：整页管理

`kinit()` 将内核结束地址 `end` 向上按 4096 字节对齐，以 `P2K(PHYSTOP)` 为上界，把可用页用头插法串入空闲链表。`FreePage` 借用空闲页开头保存下一页地址，因此无需另行分配链表节点。

`kalloc_page()` 摘取表头并返回该页地址，成功时将已分配页数加一，链表为空时返回 `NULL`；`kfree_page()` 将归还页插回表头并将页数减一。二者均由 `page_lock` 保护，保留框架提供的计数接口。

任务 3：小块管理

支持 1～2048 字节的请求。将请求大小向上取整为 8 的倍数，按“块大小 $/8-1$”选择 256 档空闲链表之一。某档为空时通过 `kalloc_page()` 取得一页，在页头用 8 字节记录档位，再将剩余空间切成等大小块串入该档，最后摘取一个块返回。

`kfree()` 通过 `PAGE_BASE(ptr)` 找到所属页，读取页头档位，再将小块插回对应链表；空指针直接返回。块起点和步长均为 8 的倍数，满足实验的对齐要求。小块链表由 `block_lock` 保护，归还的小块可复用，整页暂不退回页池，因此小块释放不减少已分配页数。

并发、复杂度与取舍

两类共享链表分别加锁，持锁分配小块时只按 `block_lock` 到 `page_lock` 的方向嵌套。忽略锁等待，整页取还、小块回收及已有块的分配均为 $O(1)$；补页最多切出 511 个块，在固定页大小下有常数上界。`kinit()` 遍历可用页，仅在启动时执行。

采用统一内存池便于各核心共享和复用，代价是锁竞争；每核独立内存池可降低竞争，但可能出现各核余量不均。本次先保证并发正确与块复用，尚未实现空页整体回收。

测试结果

使用 `cmake --build build --target qemu` 独立启动三次，原测试与统计逻辑保持不变：

次数	Total（字节）	Usage（页）	结果
1	3665447	1141	PASS
2	3665447	1144	PASS
3	3665447	1142	PASS

三次最小 Usage 为 **1141 页**，位于课程 < 1400 页档。上述三次为报告整理阶段的辅助复测，原始日志保存在 `evidence/`；本人此前手动运行同样 PASS，Usage 为 1144。过程中曾漏写取页函数的解锁语句，经检查加锁与解锁配对后补齐。

AI 教学与实现说明

使用 Codex（会话可见模型标识 GPT-6）进行教学式辅助与代码检查。我先后追问“结构体有啥用”“spinlock 是什么意思”，模型以链表取还和抢锁过程讲解，并给出关键调用示例；我根据理解手动编写、调整和运行代码。最终采用上述方案，以源码接口、对齐要求和实际 PASS 结果核对实现；本报告由工具辅助整理排版。